

# Weekly Assignment 2 – Working Report

## Deploy a Containerized Web App on CSC Rahti

### 1. Introduction

In this assignment, I deployed a simple web application as a Docker container on CSC Rahti. The main goal was to learn how to build an application from source code, run it inside a container, and then deploy that container on a platform based on OpenShift and Kubernetes. During the process, I used my GitHub repository to manage the source code, Docker Desktop to test the application locally, and the Rahti command-line tools to create and configure the application and its route.

### 2. My project files

I created a GitHub repository called “assignment-2”. It contains two important files: Dockerfile and index.html. The Dockerfile describes how the web container is created, while index.html is the web page that the container serves.

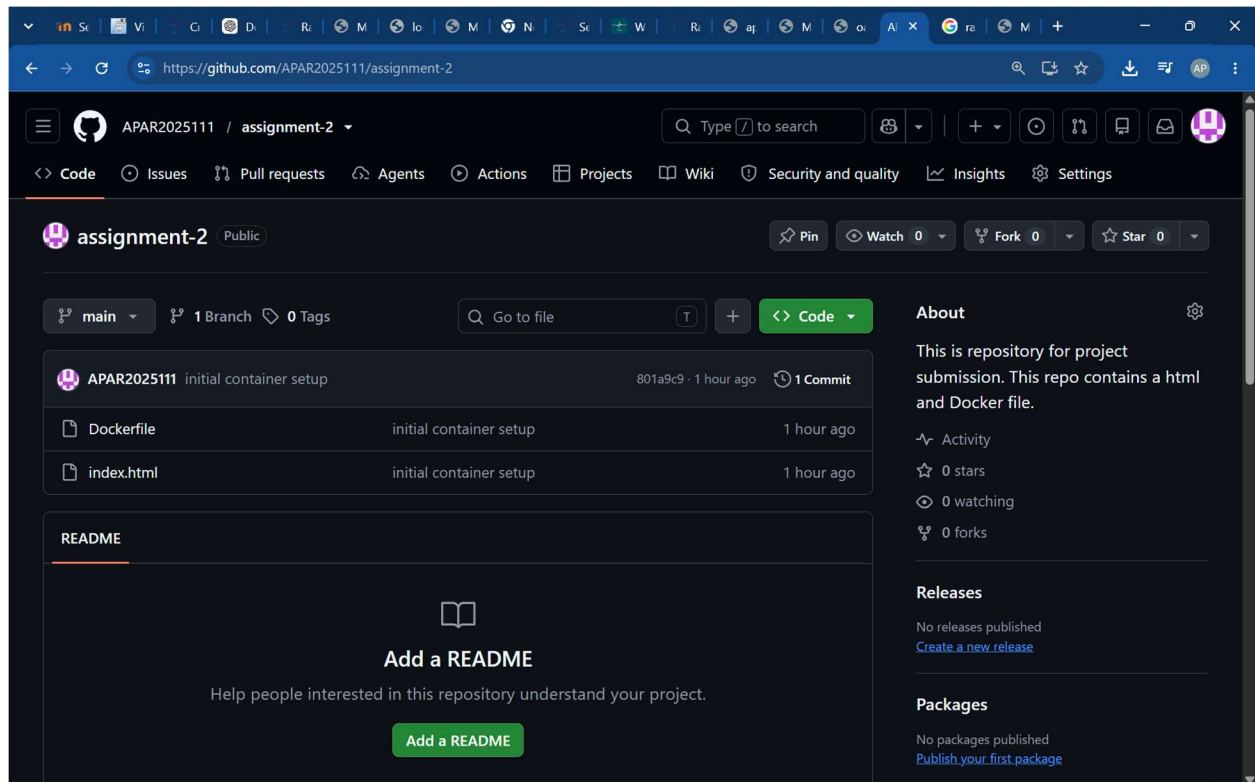


Figure 1. GitHub repository containing Dockerfile and index.html.

### 3. Checking the Docker container locally

Before deploying to Rahti, I also checked the application using Docker Desktop. The Docker image named “aps-wave-app” was available locally. I also had a running container, which helped me confirm that the application could run inside a container.

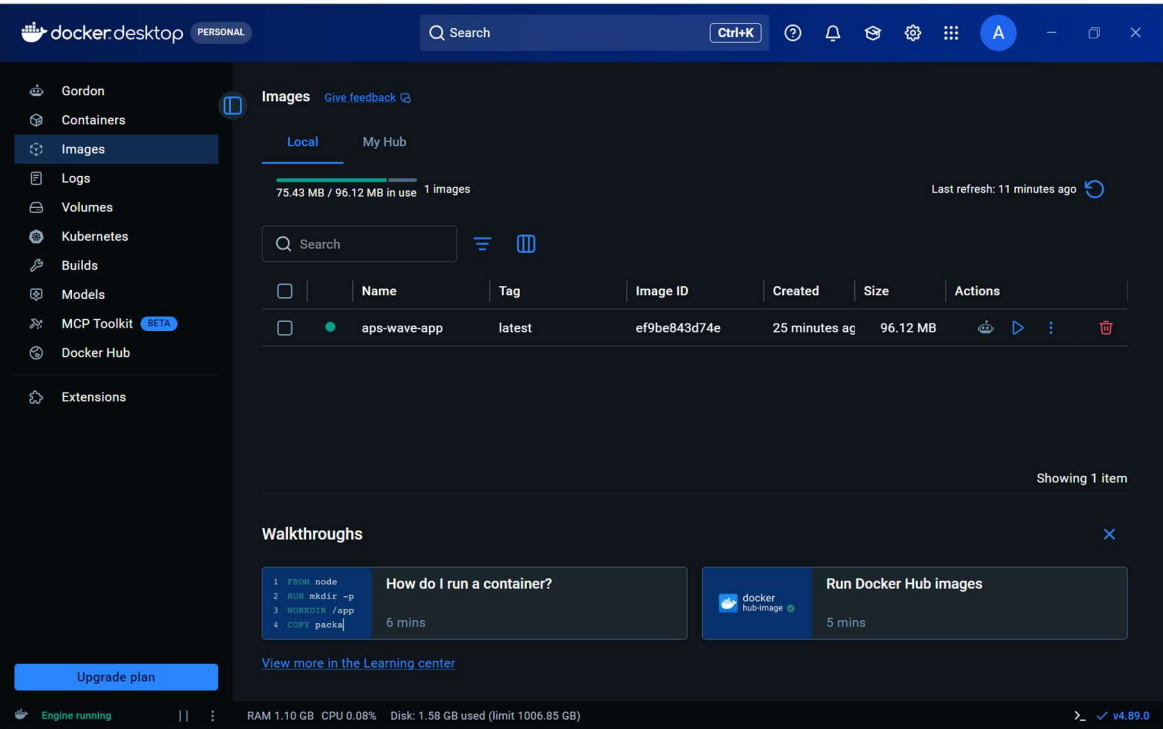


Figure 2. Docker Desktop showing the local image.

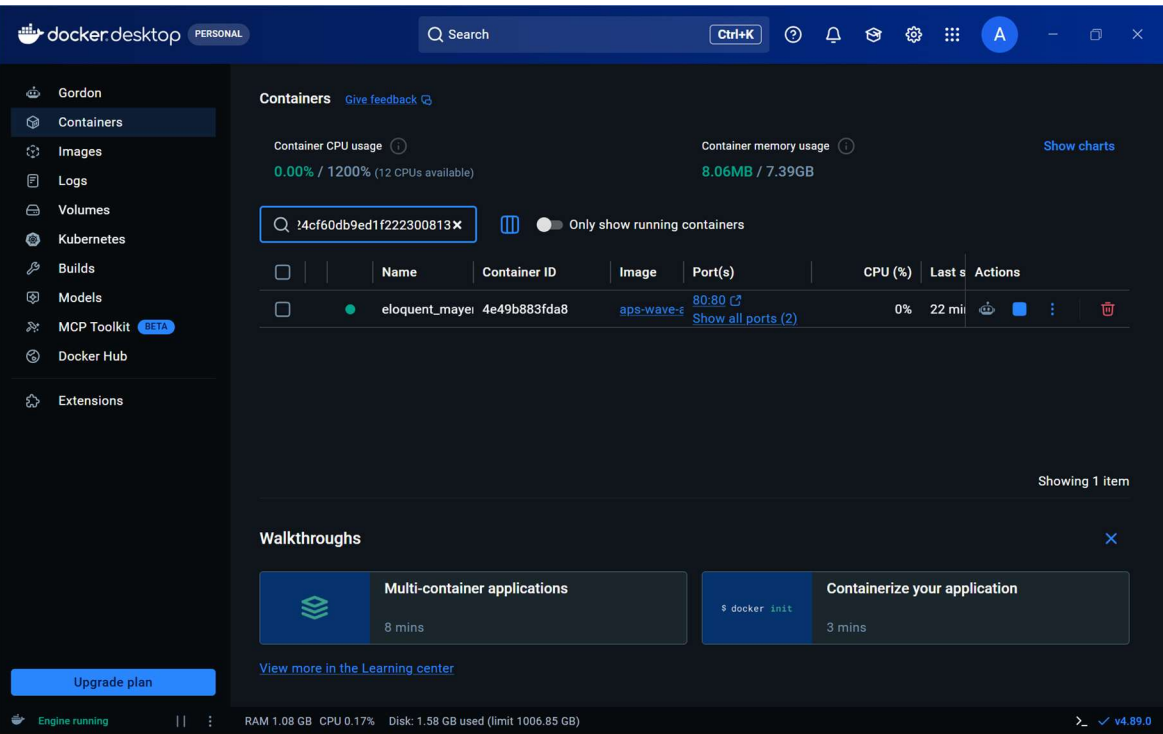
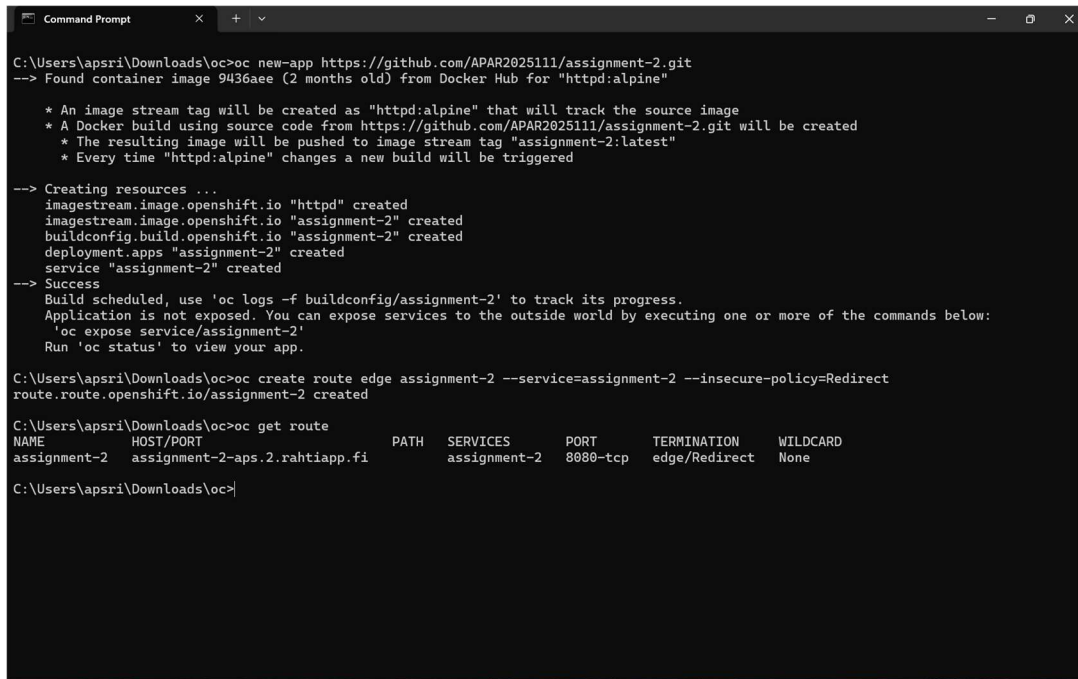


Figure 3. Docker Desktop showing the running container.

## 4. Creating the application on CSC Rahti

I then used the Rahti command line tool (oc) to create the application from my GitHub repository. The command used the repository as the source and used the httpd: alpine container image. Rahti automatically set up image stream, build configuration, deployment, and service.



```
C:\Users\apsri\Downloads\oc>oc new-app https://github.com/APAR2025111/assignment-2.git
--> Found container image 9436aee (2 months old) from Docker Hub for "httpd:alpine"

* An image stream tag will be created as "httpd:alpine" that will track the source image
* A Docker build using source code from https://github.com/APAR2025111/assignment-2.git will be created
* The resulting image will be pushed to image stream tag "assignment-2:latest"
* Every time "httpd:alpine" changes a new build will be triggered

--> Creating resources ...
imagestream.image.openshift.io "httpd" created
imagestream.image.openshift.io "assignment-2" created
buildconfig.build.openshift.io "assignment-2" created
deployment.apps "assignment-2" created
service "assignment-2" created
--> Success
Build scheduled, use 'oc logs -f buildconfig/assignment-2' to track its progress.
Application is not exposed. You can expose services to the outside world by executing one or more of the commands below:
'oc expose service/assignment-2'
Run 'oc status' to view your app.

C:\Users\apsri\Downloads\oc>oc create route edge assignment-2 --service=assignment-2 --insecure-policy=Redirect
route.route.openshift.io/assignment-2 created

C:\Users\apsri\Downloads\oc>oc get route
NAME          HOST/PORT          PATH      SERVICES    PORT      TERMINATION  WILDCARD
assignment-2  assignment-2-aps.2.rahtiapp.fi  PATH      assignment-2  8080-tcp    edge/Redirect  None

C:\Users\apsri\Downloads\oc>
```

Figure 4. Creating the application and checking the Rahti route from the command line.

## 5. Build and deployment in Rahti

After the application was created, Rahti built the container image and deployed it as a workload. The Rahti console shows the deployment named "assignment-2" and one running pod.

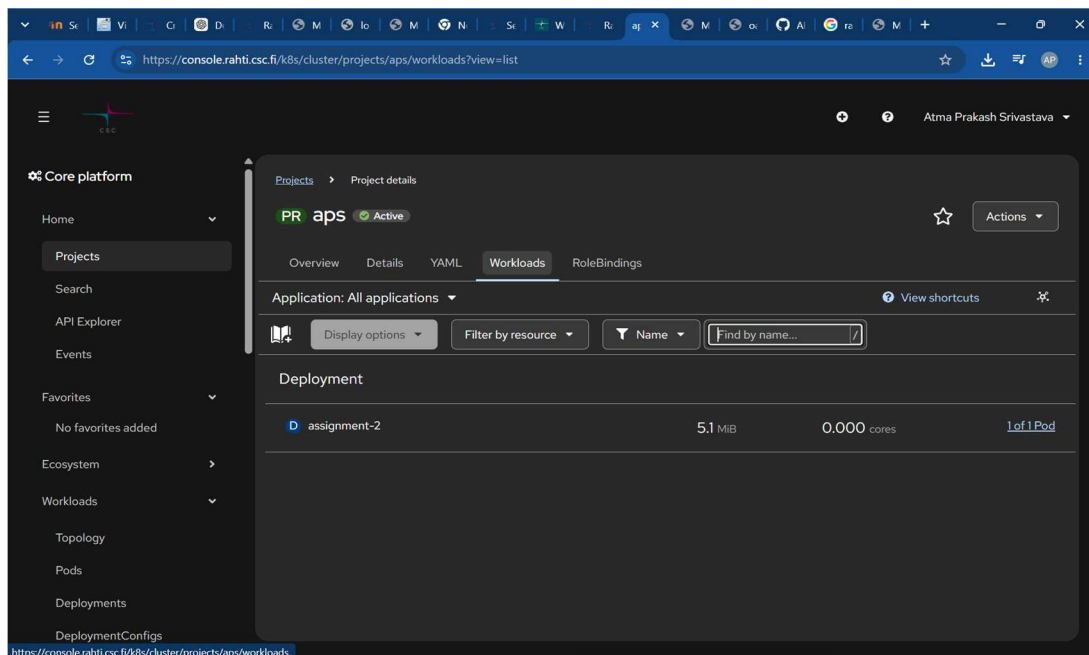


Figure 5. Rahti project showing the assignment-2 deployment and running pod.

## 6. Service and route

The deployment has a service called “assignment-2”. The service uses port 8080 and forwards traffic to the pod on port 8080. I also created a route so that the application could be accessed from outside the Rahti cluster. The route uses edge termination with redirect.

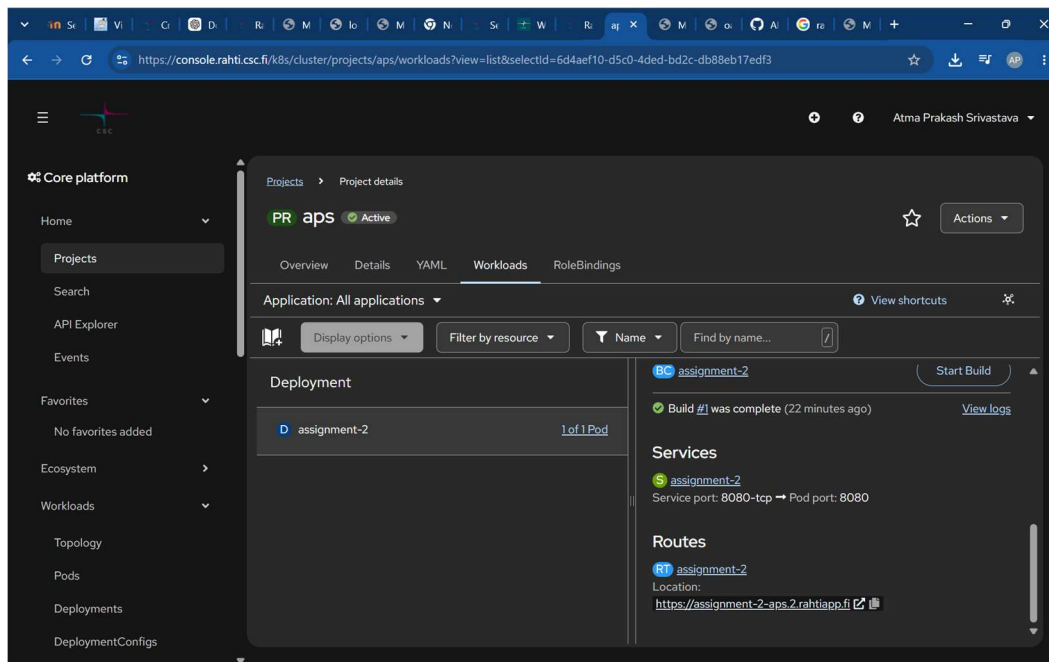


Figure 6. Rahti showing the service, port mapping and public route.

## 7. Testing the web application

Finally, I opened the public route in a web browser. The page was displayed correctly and showed “Hello from Docker!”. It also confirmed that the website was running inside an Apache Docker container.



Figure 7. The deployed website opened successfully in the browser.

## 8. Conclusion

This assignment helped me understand how a containerized web application can be deployed on a cloud platform. I learned how the GitHub source code, Dockerfile, container image, OpenShift deployment, service, and route all work together. In the end, I was able to successfully deploy the application and make it available as a public website through CSC Rahti.

10. My deployed website <https://assignment-2-aps.2.rahtiapp.fi>